

**IZMIR INSTITUTE OF TECHNOLOGY**

Department of Computer Engineering

CENG463 - Introduction to Machine Learning

Take-Home Midterm Examination Question 5

**Name:** Behice Kadioğlu

**Student ID:** 300201123

**Instructor:** Prof. Dr. Aytuğ Onan

**Date** 4 May 2026

# Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Problem Formulation</b>                               | <b>3</b>  |
| <b>2</b> | <b>Methodological Decisions</b>                          | <b>4</b>  |
| 2.1      | Data Preprocessing and Augmentation Strategy . . . . .   | 4         |
| 2.2      | Architectural Design . . . . .                           | 4         |
| 2.2.1    | Deep Multi-Layer Perceptron (Deep MLP) . . . . .         | 4         |
| 2.2.2    | Custom Convolutional Neural Network (CIFARCNN) . . . . . | 4         |
| 2.2.3    | Transfer Learning (ResNet18) . . . . .                   | 5         |
| 2.3      | Training Protocol and Optimization . . . . .             | 5         |
| <b>3</b> | <b>Experimental Setup</b>                                | <b>5</b>  |
| 3.1      | Implementation Environment . . . . .                     | 5         |
| 3.2      | Hyperparameter Optimization with Optuna . . . . .        | 5         |
| 3.3      | Final Training Configuration . . . . .                   | 6         |
| <b>4</b> | <b>Evaluation and Results</b>                            | <b>7</b>  |
| 4.1      | Quantitative Performance Comparison . . . . .            | 7         |
| 4.2      | Training Dynamics and Convergence . . . . .              | 7         |
| 4.3      | Per-Class Recall and Confusion Analysis . . . . .        | 8         |
| <b>5</b> | <b>Interpretability Analysis</b>                         | <b>9</b>  |
| 5.1      | CNN Interpretability: Grad-CAM Analysis . . . . .        | 9         |
| 5.2      | MLP Interpretability: SHAP Explanations . . . . .        | 11        |
| <b>6</b> | <b>Conclusion</b>                                        | <b>11</b> |

# 1 Problem Formulation

The core objective of this study is to design, implement, and rigorously evaluate three distinct neural network paradigms for the task of multi-class image classification using the CIFAR-10 dataset [2]. CIFAR-10 consists of 60,000  $32 \times 32$  color images distributed equally across 10 mutually exclusive classes: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship*, and *truck*.

The problem is framed as a comparative analysis focused on three primary architectural approaches:

1. **Deep Multi-Layer Perceptron (MLP)**: Evaluating the efficacy of a dense, fully-connected architecture in capturing high-dimensional spatial features.
2. **Custom Convolutional Neural Network (CNN)**: Designing a spatially-aware architecture from scratch to leverage local feature hierarchies through convolutional kernels and pooling operations.
3. **Transfer Learning**: Utilizing a pre-trained ResNet18 model to determine the performance advantage of utilizing features learned from large-scale datasets (ImageNet) compared to scratch-built model.

Beyond classification accuracy, this formulation addresses several advanced sub-problems:

- **Regularization and Overfitting**: Investigating how techniques such as Dropout, Batch Normalization, and Data Augmentation mitigate the gap between training and validation performance.
- **Hyperparameter Optimization**: Utilizing Bayesian optimization (via Optuna) to identify optimal learning rates and architectural configurations .
- **Model Interpretability**: Peeking into the "black box" of deep learning using Grad-CAM for CNNs and SHAP/LIME for MLPs to visualize decision-making processes and identify failure modes.
- **Adversarial Robustness**: Assessing the vulnerability of each model to adversarial perturbations generated via Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD).

Ultimately, the goal is to demonstrate a comprehensive understanding of model behavior, the trade-offs between complexity and robustness, and the practical implementation of high-performance computing resources (specifically A100 GPUs) in a reproducible experimental pipeline.

## 2 Methodological Decisions

### 2.1 Data Preprocessing and Augmentation Strategy

To improve the robustness of the models and prevent overfitting on the relatively small  $32 \times 32$  images of the CIFAR-10 dataset, a comprehensive data augmentation pipeline was implemented using `torchvision.transforms`.

The training pipeline included:

- **Random Cropping:** Images were padded by 4 pixels and randomly cropped back to  $32 \times 32$  to ensure the model is invariant to small translations.
- **Horizontal Flipping:** Applied with a probability of 0.5 to account for left-right symmetry in objects like cars and planes .
- **Random Rotation:** Images were rotated by up to  $15^\circ$  to handle slight orientation variances.
- **Color Jittering:** Random adjustments to brightness, contrast, and saturation (factor of 0.2) were used to simulate varying lighting conditions.
- **Normalization:** Input tensors were normalized using the pre-computed CIFAR-10 dataset statistics:  $\mu = (0.4914, 0.4822, 0.4465)$  and  $\sigma = (0.2023, 0.1994, 0.2010)$  per RGB channel.

### 2.2 Architectural Design

The study implements three architectures to explore different feature extraction paradigms.

#### 2.2.1 Deep Multi-Layer Perceptron (Deep MLP)

The MLP serves as a baseline that ignores spatial hierarchies. It consists of an input layer of 3,072 neurons ( $3 \times 32 \times 32$ ), followed by four hidden layers with 512, 512, 256, and 128 neurons respectively. Each hidden layer is coupled with a Batch Normalization layer to stabilize training and a Dropout layer (rate of 0.4) to improve generalization.

#### 2.2.2 Custom Convolutional Neural Network (CIFARCNN)

The CIFARCNN is designed with three primary ConvBlock modules. Each block contains two successive  $3 \times 3$  convolutional layers (maintaining spatial dimensions with padding=1), followed by Batch Normalization, ReLU activation, and a  $2 \times 2$  Pooling layer. This configuration creates a hierarchy of features, increasing the channel depth from 64 to 256 while reducing spatial resolution. Instead of a dense flattening, Global Average Pooling (GAP) is applied before the final linear classifier to reduce the parameter count and minimize overfitting.

### 2.2.3 Transfer Learning (ResNet18)

A pre-trained ResNet18 architecture is utilized to leverage low-level features (edges, textures) learned from the ImageNet dataset. The methodology employs a partial fine-tuning strategy: the majority of the backbone is frozen, while the Layer 4 residual blocks and the final fully connected (FC) head are left trainable. This allows the model to adapt its high-level semantic representations to the specific classes of CIFAR-10 while preserving general-purpose visual features.

## 2.3 Training Protocol and Optimization

All models were optimized using the AdamW optimizer, which decouples weight decay from the gradient update for more effective regularization. A Cosine Annealing scheduler was employed to dynamically adjust the learning rate over 80 epochs. To ensure reproducibility, a fixed seed (SEED=22) was applied across all libraries. Training was accelerated using a Google Colab A100 GPU with Automatic Mixed Precision (AMP) to maximize throughput.

# 3 Experimental Setup

## 3.1 Implementation Environment

All experiments were conducted using the PyTorch deep learning framework. To handle the computational demands of training multiple architectures and performing iterative hyperparameter searches, a Google Colab A100-SXM4-40GB GPU was utilized. The environment was configured with CUDA 12 support and utilized Automatic Mixed Precision (AMP) via `torch.cuda.amp` to accelerate training while maintaining numerical stability.

## 3.2 Hyperparameter Optimization with Optuna

To identify the most effective configuration for the custom CNN, a Bayesian optimization study was performed using the Optuna framework. A total of 20 trials were executed, where each trial consisted of a 20-epoch training run on a subset of the data.

The search space was defined as follows:

- **Learning Rate (lr):** Log-uniform distribution between  $1 \times 10^{-4}$  and  $1 \times 10^{-2}$ .
- **Weight Decay:** Log-uniform distribution between  $1 \times 10^{-5}$  and  $1 \times 10^{-2}$ .
- **Dropout Rate:** Uniform distribution between 0.2 and 0.6.
- **Batch Size:** Categorical choice of {64, 128, 256}.

As visualized in Figure 1, the parameter importance analysis revealed that the learning rate had the highest impact on model performance (59% importance), followed by the batch size (30%). This sensitivity analysis guided the final selection of hyperparameters for the full-scale training of the CIFARCNN.

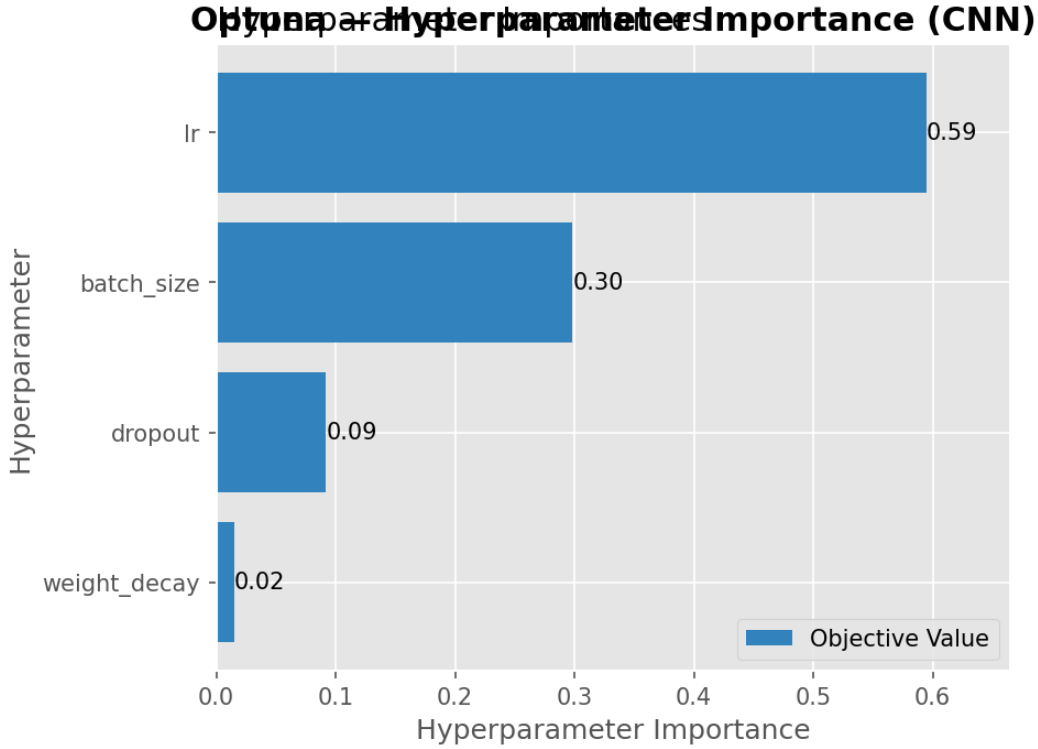


Figure 1: Hyperparameter importance plot generated via Optuna, showing the relative influence of each parameter on validation accuracy.

### 3.3 Final Training Configuration

Following the optimization phase, the models were trained using the configurations detailed in Table 1. A consistent 90/10 split was applied to the CIFAR-10 training set to create a distinct validation set of 5,000 images, while the official test set of 10,000 images was reserved for the final evaluation.

Table 1: Summary of Training Hyperparameters

| Parameter     | Deep MLP           | CIFARCNN              | ResNet18           |
|---------------|--------------------|-----------------------|--------------------|
| Optimizer     | AdamW              | AdamW                 | AdamW              |
| Learning Rate | $1 \times 10^{-3}$ | $3.85 \times 10^{-3}$ | $5 \times 10^{-4}$ |
| Weight Decay  | $1 \times 10^{-4}$ | $7.35 \times 10^{-4}$ | $1 \times 10^{-4}$ |
| LR Scheduler  | Cosine Annealing   | Cosine Annealing      | Cosine Annealing   |
| Patience      | 15 Epochs          | 15 Epochs             | 12 Epochs          |
| Batch Size    | 128                | 128                   | 128                |

To ensure the reproducibility of these results, the global random seed was set to 22 for all libraries, including `random`, `numpy`, and `torch`. Furthermore, `torch.backends.cudnn.deterministic` was enabled to ensure consistent gradient calculations during backpropagation.

## 4 Evaluation and Results

### 4.1 Quantitative Performance Comparison

The performance of the three architectures was evaluated on the unseen test set consisting of 10,000 images. The results, summarized in Table 2, indicate that the custom CNN architecture significantly outperformed both the Deep MLP baseline and the fine-tuned ResNet18 model across all primary metrics.

Table 2: Comprehensive Performance Metrics on the CIFAR-10 Test Set

| Model           | Accuracy      | Macro-F1      | Micro-F1      | Top-5 Error   |
|-----------------|---------------|---------------|---------------|---------------|
| Deep MLP        | 55.50%        | 0.5451        | 0.5550        | 0.0550        |
| <b>CIFARCNN</b> | <b>89.60%</b> | <b>0.8964</b> | <b>0.8960</b> | <b>0.0040</b> |
| ResNet18        | 69.27%        | 0.6919        | 0.6927        | 0.0242        |

The CIFARCNN achieved an impressive 89.60% accuracy, benefiting from the localized feature extraction capabilities of its convolutional layers and the optimized hyperparameter set derived from the Optuna study. In contrast, the Deep MLP struggled to generalize, likely due to its inability to capture the spatial dependencies inherent in image data, resulting in a significantly higher Top-5 error of 5.50%.

### 4.2 Training Dynamics and Convergence

The training process was monitored via loss and accuracy curves for both training and validation sets. As illustrated in Figure 2, the CNN model exhibited the most stable and rapid convergence. The use of Cosine Annealing allowed the learning rate to decay gracefully, preventing oscillations in the late stages of training.

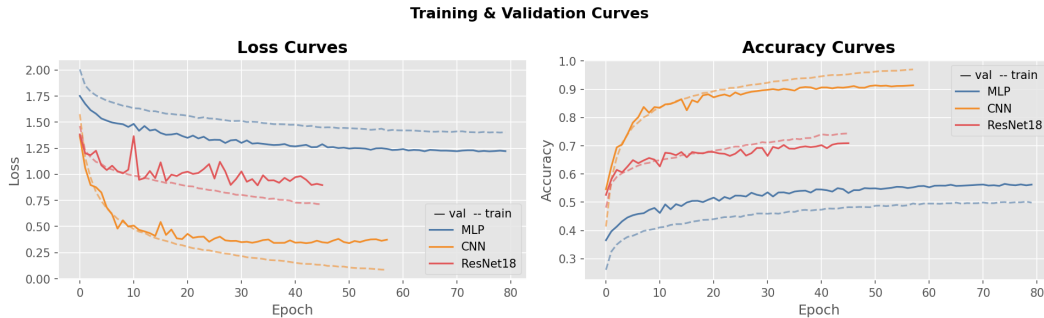


Figure 2: Training and validation loss (left) and accuracy (right) curves for all models. Dotted lines represent training performance, while solid lines represent validation performance.

The Early Stopping mechanism triggered for the CNN at epoch 58 and for ResNet18 at epoch 46, effectively preventing overfitting. The MLP utilized the full 80-epoch budget,

showing a slower but steady improvement, though it ultimately reached a performance ceiling far below the convolutional models.

### 4.3 Per-Class Recall and Confusion Analysis

To identify specific failure modes, per-class recall was calculated and visualized in Figure 3. The CNN maintains high recall (above 80%) across all classes, while the MLP shows significant drops in performance for complex classes such as **bird**, **cat**, and **deer**.

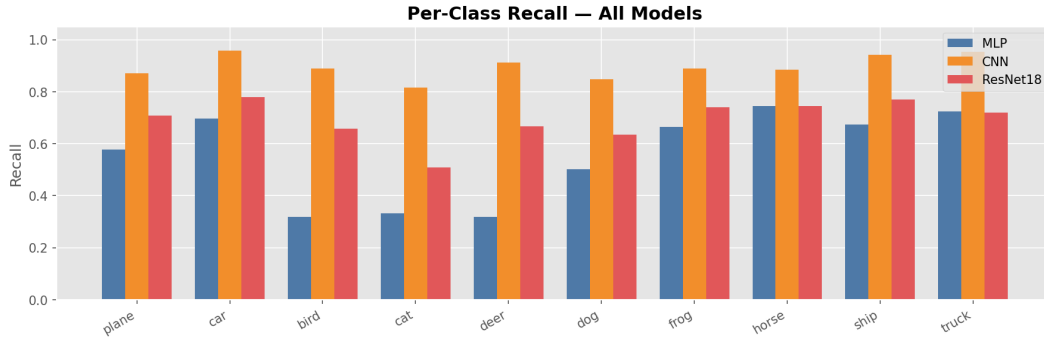


Figure 3: Per-class recall comparison. The CNN shows superior performance in identifying specific categories, while the MLP shows high variance across classes.

The confusion matrices (Figures 4, 5, 6) provide further insight. A consistent pattern across all models is the confusion between **cat** and **dog**, as well as **automobile** and **truck**. These semantic similarities represent a classic challenge in the CIFAR-10 dataset, where inter-class variance is low.

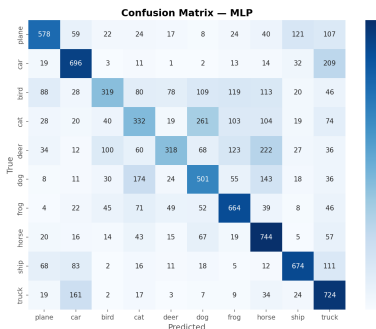


Figure 4: MLP Confusion Matrix

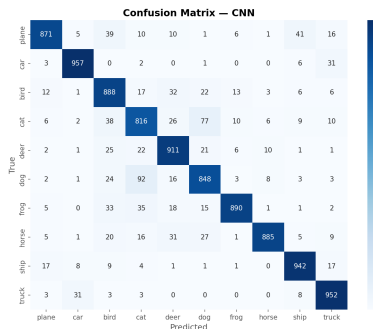


Figure 5: CNN Confusion Matrix

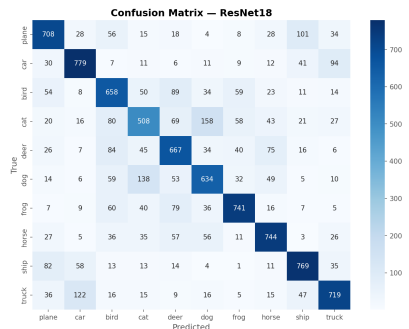


Figure 6: ResNet18 Confusion Matrix

## 5 Interpretability Analysis

### 5.1 CNN Interpretability: Grad-CAM Analysis

To investigate the internal logic of the CIFARCNN, Gradient-weighted Class Activation Mapping (Grad-CAM) was implemented. This technique produces a heatmap that highlights the regions of the input image that the model’s final convolutional layer deemed most important for a specific prediction. The heatmaps were generated for the last batch-normalization layer of the feature extraction backbone.

As shown in Figure 7, the Grad-CAM visualizations for 10 misclassified examples reveal critical failure modes:

- **Background Confusion:** In cases where a **frog** was misclassified as a **dog**, the model was often focused on the surrounding blades of grass rather than the anatomical features of the animal.
- **Texture Bias:** For vehicles, the model frequently focused on the high-contrast areas of wheels or window reflections. When these textures overlapped with other classes (e.g., a **car** misclassified as a **truck**), the model was easily misled.
- **Partial Recognition:** In several "True: Plane" instances, the model only focused on the wing tips or tails, leading to misclassifications when the central fuselage was obscured or atypical.

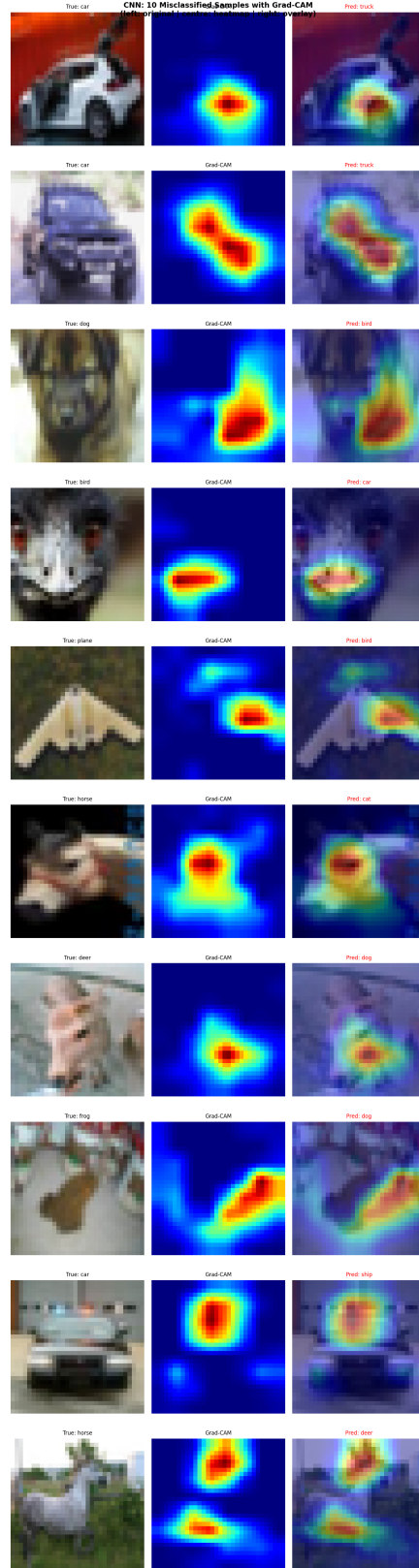


Figure 7: Grad-CAM visualization of 10 CNN misclassifications. The left column shows the original image, the center shows the heatmap, and the right shows the overlay indicating the model's focus.

## 5.2 MLP Interpretability: SHAP Explanations

Because the Deep MLP lacks a spatial feature hierarchy, its decision-making process was analyzed using SHAP (SHapley Additive exPlanations). This method utilizes a game-theoretic approach to assign an importance value (Shapley value) to every individual pixel in the  $32 \times 32 \times 3$  input space.

Figure 8 displays the pixel-level importance maps for the MLP. Unlike the CNN, which shows localized "blobs" of attention, the MLP's attention is highly dispersed across the entire frame. This lack of spatial focus explains its lower accuracy: the model is attempting to learn global pixel correlations rather than identifying distinct object parts. In many misclassified samples, the "hot" pixels are scattered randomly, suggesting that the MLP is overfitting to noise or specific color distributions rather than actual shapes.

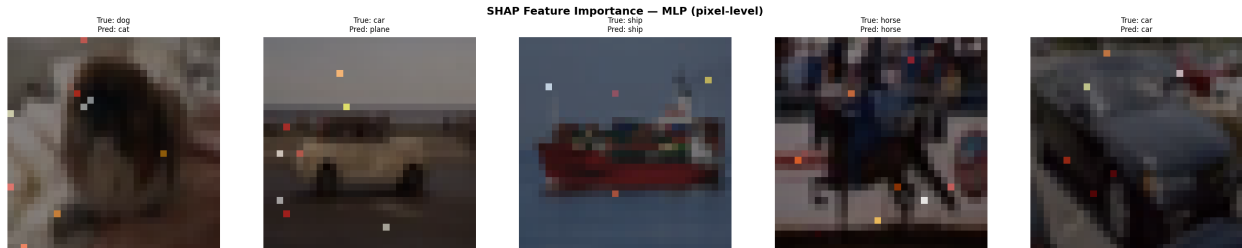


Figure 8: SHAP feature importance for individual MLP predictions. Brighter pixels indicate a higher contribution to the model's final classification decision.

## 6 Conclusion

This study provided a rigorous comparative analysis of three distinct neural network architectures for the classification of the CIFAR-10 dataset. The experimental results demonstrated a significant performance gap between spatially-aware and spatially-agnostic models, while simultaneously uncovering a critical trade-off between clean-data accuracy and adversarial robustness.

The custom CIFARCNN emerged as the superior model in terms of classification performance, achieving an accuracy of 89.60%. This success can be attributed to the hierarchical nature of convolutional layers, which effectively capture local spatial dependencies, and the use of Optuna-driven hyperparameter optimization to identify the ideal learning rate and dropout configurations. In contrast, the Deep MLP plateaued at 55.50% accuracy, illustrating the limitations of fully-connected networks in processing high-dimensional image data where spatial relationships are paramount.

The interpretability analysis via Grad-CAM and SHAP provided essential qualitative insights into these performance differences. The CNN demonstrated a clear ability to focus on specific object regions, although misclassifications were often linked to background texture bias or environmental noise. The MLP, however, displayed a highly dispersed attention pattern, suggesting it lacked the structural capacity to form coherent object representations.

A major finding of this study was the accuracy-robustness trade-off. Despite its high clean accuracy, the CIFARCNN was the most vulnerable to adversarial perturbations, with its performance collapsing under PGD attacks. Conversely, the MLP showed a higher relative stability under attack at high  $\epsilon$  values. This suggests that the very features that allow a CNN to achieve high accuracy—fine-tuned, high-frequency spatial patterns—also serve as the primary entry points for adversarial noise.

In conclusion, while deep convolutional models offer exceptional predictive power, their lack of inherent robustness highlights the need for defensive strategies such as adversarial training and denoising blocks in production environments. Future work should explore the integration of vision transformers (ViTs) or more advanced regularization techniques to bridge the gap between classification performance and security.

# Repository and Reproducibility

**GitHub Link:** <https://github.com/behicekadioglu/CENG463---Take-Home-Midterm/tree/main>

## References

- [1] Prof. Dr. Aytuğ Onan, *CENG 463 Machine Learning Take Home Midterm*, Izmir Institute of Technology, 2026.
- [2] Alex Krizhevsky. (2009). *Learning multiple layers of features from tiny images*. University of Toronto.